Ministerul Educaţiei şi Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică şi Microelectronică

Laboratory Work No. 3:

Polyalphabetic Ciphers

Task 3.1 — Playfair Cipher

Course: Cryptography and Security

Elaborated: st. gr. FAF-241 Lungu Ilie

Verified: asist. univ. *Zaica Maia*

Chişinău – 2026

Contents

1 SHORT THEORY **2**
- 1.1 Text preparation . . . 2
- 1.2 Encryption matrix . . . 2
- 1.3 Encryption and decryption rules . . . 2

2 TASK AND DESIGN DECISIONS **3**
- 2.1 Task statement . . . 3
- 2.2 Alphabet and matrix size . . . 3

3 IMPLEMENTATION **3**
- 3.1 Verification against the handout example . . . 4

4 RESULTS **4**
- 4.1 Test 1 . . . 4
- 4.2 Test 2: repeated letters . . . 4
- 4.3 Test 3: merged letter and filler collisions . . . 5
- 4.4 Input validation . . . 5

5 CONCLUSIONS **5**

A Source code: `playfair.py` **6**

1 SHORT THEORY

The weakness of monoalphabetic ciphers, studied in the previous laboratory work, is that their ciphertext reproduces the letter-frequency distribution of the plaintext language. A cipher is cryptographically stronger when its distribution is as flat as possible, so that it gives the cryptanalyst no information. One way to flatten it is to let the substitution of a letter change along the text, depending on its position or context, which also increases the number of possible keys considerably.

The Playfair cipher, invented by Charles Wheatstone in 1854 and popularized by Lord Lyon Playfair, takes this idea further: it encrypts *pairs of letters* (digraphs) instead of single letters. With an alphabet of n symbols there are n^2 digraphs, so the frequency table an attacker must analyse is far larger and flatter than a table of n single letters. For this reason Playfair was used by the British army in the Boer War and, in modified forms, in both world wars. It is obsolete by modern standards, but it is one of the first ciphers that work on blocks of text, which makes it a good introduction to modern block ciphers.

Playfair encryption consists of three steps: (a) preparing the text, (b) building the encryption matrix from the key, and (c) transforming the digraphs with the matrix.

1.1 Text preparation

The message is written in upper case, without spaces or punctuation, and split into pairs of letters. If a pair would consist of two identical letters, a rarely used letter (X, or Z if the repeated letter is X) is inserted between them. If the number of letters is odd, the same filler letter is appended at the end.

1.2 Encryption matrix

The key is written row by row into the matrix, skipping any letter that has already appeared. The remaining letters of the alphabet follow in alphabetical order. If the alphabet does not fill the matrix exactly, a rare letter is merged with another one (in the English version, J is replaced by I).

1.3 Encryption and decryption rules

For every pair of letters (a, b) located at (r_1, c_1) and (r_2, c_2) in a matrix with R rows and C columns:

1. **Different row and column** (rectangle rule): each letter is replaced by the letter on its own row and in the column of the other letter: (r_1, c_2) and (r_2, c_1). Decryption is identical.

2. **Same row**: each letter is replaced by the letter to its right, $(r, (c + 1) \bmod C)$. Decryption uses the letter to the left, $(r, (c - 1) \bmod C)$.

3. **Same column**: each letter is replaced by the letter below it, $((r + 1) \bmod R, c)$. Decryption uses the letter above it, $((r - 1) \bmod R, c)$.

2 TASK AND DESIGN DECISIONS

2.1 Task statement

Task 3.1 requires a program implementing the Playfair cipher for messages in Romanian (31 letters). The following requirements were implemented:

- only the characters `A`–`Z` and `a`–`z` are accepted; for any other character the program suggests the correct range;
- the key must contain at least 7 letters;
- the user chooses the operation (encryption or decryption), enters the key and the message or ciphertext, and obtains the ciphertext or the decrypted message;
- adding spaces to the decrypted text is done manually.

The program is written in Python 3 (file `playfair.py`, listed in Appendix A).

2.2 Alphabet and matrix size

The Romanian alphabet has 31 letters: A, Ă, Â, B, C, D, E, F, G, H, I, Î, J, K, L, M, N, O, P, Q, R, S, Ş, T, Ţ, U, V, W, X, Y, Z. Since 31 is a prime number, it cannot fill a rectangular matrix. Following the handout (the rarest letter may be removed and restored by the reader), the letter Q, which occurs in Romanian only in foreign words, is merged with K. The remaining 30 letters form a matrix with 5 rows and 6 columns. The filler letters are X and, when the repeated letter is itself X, Z.

Because the plaintext is restricted to `A`–`Z`, the letters Ă, Â, Î, Ş and Ţ never occur in a message. They still occupy cells of the matrix, so they can appear in the *ciphertext*. For this reason the decryption mode also accepts these five letters in the cryptogram, while the plaintext, the key and the message are checked strictly against the range `A`–`Z`, `a`–`z`.

3 IMPLEMENTATION

The program is organized in small functions.

- `normalize` removes whitespace, converts the text to upper case and applies the merge Q→K.
- `validate_plain` and `validate_cipher` check the characters and, on error, print the allowed range.
- `build_matrix` writes the key without repeated letters followed by the rest of the alphabet into the 5×6 matrix.
- `prepare` splits the text into digraphs and inserts the filler letters.
- `_transform` applies the three rules; a single parameter selects the direction (`+1` encrypts, `-1` decrypts), so encryption and decryption share the same code.
- `main` implements the menu: it repeatedly asks for the operation, the key (minimum 7 letters), and the message or cryptogram.

The decrypted text is printed without spaces and still contains the filler letters, because the handout requires the final step (restoring spaces and removing padding) to be done manually.

3.1 Verification against the handout example

To check the correctness of the rules, the same code was run in the original configuration of the handout (English alphabet, J replaced by I, a 5×5 matrix) with the key `FIRST AMEND`. The matrix obtained is identical to the one in the handout:

F	I	R	S	T
A	M	E	N	D
B	C	G	H	K
L	O	P	Q	U
V	W	X	Y	Z

Decrypting the beginning of the handout cryptogram `OWEHEGRYTYNQBVOAEMGDMQ...` produced `CONGRESXSZSHALLMMAKENO...`, which coincides, letter by letter, with the prepared text printed in the handout (`CO NG RE SX SZ SH AL LM MA KE NO LA WR ES PE CT IN GA NE ST...`). This confirms the row, column and rectangle rules and their inverses.

4 RESULTS

Three tests were run with the Romanian configuration (5×6 matrix).

4.1 Test 1

Key: `SECURITATE`; message: *Limba romana este comoara noastra.*

S	E	C	U	R	I
T	A	Ă	Â	B	D
F	G	H	Î	J	K
L	M	N	O	P	Ş
Ţ	V	W	X	Y	Z

After removing the repeated letters, the key becomes `SECURITA` (10 letters, 8 distinct), so the matrix starts with S, E, C, U, R, I, T, A. Prepared digraphs: `LI MB AR OM AN AE ST EC OM OA RA NO AS TR AX` (the filler X completes the odd-length text).

Ciphertext:	`ŞSPABEPNĂMGATFCUPNMÂEBOPTEBSÂV`
Decrypted:	`LIMBAROMANAESTECOMOARANOASTRAX`

After removing the filler and restoring the spaces manually, the message is *Limba romana este comoara noastra.*

4.2 Test 2: repeated letters

Key: `CRIPTOGRAFIE`; message: *Aleea este linistita.* The double letter EE falls inside one digraph, so a filler is inserted between the two letters.

C	R	I	P	T	O
G	A	F	E	Ă	Â
B	D	H	Î	J	K
L	M	N	S	Ş	Ţ
U	V	W	X	Y	Z

The pairing `AL EE A...` becomes `AL EX EA...`, and the odd length of the text requires a final filler. Prepared digraphs: `AL EX EA ES TE LI NI ST IT AX`.

Ciphertext:	`GMÎPĂFÎXPĂNCWFŞPPOEV`
Decrypted:	`ALEXEAESTELINISTITAX`

4.3 Test 3: merged letter and filler collisions

Key: `LUCRARE`; message: *Quasar xx*. The letter Q is replaced by K, and the repeated X is separated with the alternative filler Z. Prepared digraphs: `KU AS AR XZ XZ`.

Ciphertext:	`IARŞEAYŢYŢ`
Decrypted:	`KUASARXZXZ`

The decrypted text shows K instead of Q and the fillers X/Z; the reader restores them manually, as allowed by the handout.

4.4 Input validation

Invalid input produces a hint with the correct range, and the program asks again instead of stopping. Example session (user input after the prompt):

```
Operation [1 = encrypt, 2 = decrypt, 0 = exit]: 1
Key (min. 7 letters): abc
  Error: the key must have at least 7 letters.
Key (min. 7 letters): CRIPTOGRAFIE
Message: Limba romana 2026
  Error: Only the Latin letters 'A'-'Z' and 'a'-'z' are allowed
  (spaces are ignored). Remove digits, punctuation and diacritics.
Message: Limba romana este comoara noastra
Ciphertext: NCLDDARFMFPGPRRÂADIEMOIEV
```

When decrypting, a cryptogram with an odd number of letters or with symbols outside the alphabet is rejected with an explanatory message as well.

5 CONCLUSIONS

The Playfair cipher was implemented for Romanian text using a 5×6 matrix built from the 31-letter alphabet, with Q merged into K. The program validates the input range and the minimum key length, handles repeated letters and odd-length messages with the fillers X and Z, and uses the same function for encryption and decryption. The implementation was checked against the example in the handout, and the three Romanian tests decrypted back to the prepared plaintext.

Compared with a monoalphabetic cipher, Playfair hides single-letter frequencies: the same plaintext letter is encrypted differently depending on its neighbour. The cipher is nevertheless weak by today's standards, because it preserves the frequency structure of digraphs and has a small key space compared with modern ciphers, and the same plaintext digraph always gives the same ciphertext digraph. Its limitations are visible in the implementation as well: the merged letter Q→K and the filler letters change the text and must be corrected by hand when reading the result.

A Source code: `playfair.py`

```
#!/usr/bin/env python3
"""Lab 3, Task 3.1 -- Playfair cipher over the Romanian alphabet (31 letters)."""

import re

# Romanian alphabet: 31 letters.
RO_ALPHABET = "AĂÂBCDEFGHIÎJKLMNOPQRSTUVWXYZ"

# 31 is prime, so it cannot fill a rectangular matrix. Following the handout
# (drop the rarest letter), Q is merged with K -> 30 symbols -> 5x6 matrix.
MERGE = {"Q": "K"}
ROWS, COLS = 5, 6
MATRIX_ALPHABET = "".join(c for c in RO_ALPHABET if c not in MERGE)

FILLER, FILLER_ALT = "X", "Z"      # rare letters used as padding
MIN_KEY_LEN = 7
VALID_TEXT = re.compile(r"[A-Za-z]+")


def normalize(text, alphabet=MATRIX_ALPHABET, merge=MERGE):
    """Upper-case the text and apply the letter merges (Q -> K)."""
    text = "".join(text.split()).upper()
    return "".join(merge.get(c, c) for c in text)


def validate_plain(text):
    """Plaintext/key: only A-Z, a-z (spaces are removed first)."""
    compact = "".join(text.split())
    if not compact or not VALID_TEXT.fullmatch(compact):
        raise ValueError("Only the Latin letters 'A'-'Z' and 'a'-'z' are "
                         "allowed (spaces are ignored). Remove digits, "
                         "punctuation and diacritics.")
    return compact


def validate_cipher(text):
    """Ciphertext may also contain  Â Î   (they sit in the matrix)."""
    compact = "".join(text.split()).upper()
    if not compact or any(c not in MATRIX_ALPHABET for c in compact):
        raise ValueError("The ciphertext may contain only letters of the "
                         "Romanian alphabet ('A'-'Z', 'a'-'z', , Â, Î, , "
                         "), without 'Q'.")
    return compact


def build_matrix(key, alphabet=MATRIX_ALPHABET, cols=COLS):
    """Key without repeated letters first, then the rest of the alphabet."""
    seen = []
    for c in normalize(key) + alphabet:
        if c in alphabet and c not in seen:
            seen.append(c)
    return [seen[i:i + cols] for i in range(0, len(seen), cols)]


def prepare(text):
    """Split into digraphs; insert a filler between equal letters and
    pad an odd-length text with a filler."""
    pairs, i = [], 0
    while i < len(text):
        a = text[i]
        b = text[i + 1] if i + 1 < len(text) else None
        if b is None or a == b:
            pairs.append(a + (FILLER if a != FILLER else FILLER_ALT))
            i += 1
        else:
            pairs.append(a + b)
            i += 2
    return pairs


def _positions(matrix):
    return {c: (r, k) for r, row in enumerate(matrix) for k, c in enumerate(row)}


```

```
def _transform(pairs, matrix, step):
    """step = +1 encrypts, step = -1 decrypts."""
    pos = _positions(matrix)
    rows, cols = len(matrix), len(matrix[0])
    out = []
    for a, b in pairs:
        (r1, c1), (r2, c2) = pos[a], pos[b]
        if r1 == r2:                                    # same row
            out.append(matrix[r1][(c1 + step) % cols] +
                       matrix[r2][(c2 + step) % cols])
        elif c1 == c2:                                  # same column
            out.append(matrix[(r1 + step) % rows][c1] +
                       matrix[(r2 + step) % rows][c2])
        else:                                           # rectangle
            out.append(matrix[r1][c2] + matrix[r2][c1])
    return out


def encrypt(message, key):
    matrix = build_matrix(key)
    pairs = prepare(normalize(message))
    return "".join(_transform(pairs, matrix, +1))


def decrypt(cipher, key):
    matrix = build_matrix(key)
    text = normalize(cipher)
    pairs = [text[i:i + 2] for i in range(0, len(text), 2)]
    return "".join(_transform(pairs, matrix, -1))


def show(matrix):
    return "\n".join(" ".join(row) for row in matrix)


def read_key():
    while True:
        try:
            key = validate_plain(input("Key (min. %d letters): " % MIN_KEY_LEN))
        except ValueError as e:
            print("  Error:", e)
            continue
        if len(key) < MIN_KEY_LEN:
            print("  Error: the key must have at least %d letters." % MIN_KEY_LEN)
            continue
        return key


def main():
    while True:
        op = input("Operation [1 = encrypt, 2 = decrypt, 0 = exit]: ").strip()
        if op == "0":
            break
        if op not in ("1", "2"):
            print("  Error: choose 1, 2 or 0.")
            continue
        key = read_key()
        print("Matrix:\n" + show(build_matrix(key)))
        while True:
            try:
                if op == "1":
                    msg = validate_plain(input("Message: "))
                    print("Ciphertext:", encrypt(msg, key))
                else:
                    msg = validate_cipher(input("Ciphertext: "))
                    if len(msg) % 2:
                        raise ValueError("The ciphertext must have an even "
                                         "number of letters.")
                    print("Decrypted text:", decrypt(msg, key))
                break
            except ValueError as e:
                print("  Error:", e)


if __name__ == "__main__":
    main()
```